Experiment 1: Word Count using MapReduce in Python

MapReduce Laboratory

Aryan Awasthi
Roll No.: 102315182
Subgroup: 402

1. Aim

To write a word count program in Python using the MapReduce programming model, and to run it with Hadoop Streaming.

2. Theory

MapReduce is a programming model for processing large data sets in parallel on a cluster. A job has three phases:

Phase	What happens
Map	Each mapper reads one split of the input and turns every record into key-value pairs.
Shuffle and Sort	The framework groups the pairs by key, sorts them, and sends all values of one key to the same reducer.
Reduce	Each reducer receives a key with all its values and combines them into the final result.

Hadoop Streaming lets the mapper and reducer be any program that reads lines from standard input and writes key TAB value lines to standard output. This is how Python programs run on Hadoop.

Word count is the standard first MapReduce program:

- **Map:** for every word in a line, emit (word, 1).
- **Shuffle and Sort:** all (word, 1) pairs for the same word arrive together at one reducer.
- **Reduce:** add up the 1s for each word to get how many times it appears.
- **Combiner:** the reducer logic can also run on each mapper's output (addition is associative and commutative), so fewer pairs are sent across the network.

3. Algorithm

Mapper

1. Read each line from standard input.
2. Convert it to lower case and split it into words (letters, digits and apostrophes).
3. For every word, print word TAB 1.

Reducer

1. Read the sorted word TAB count lines.
2. While the word stays the same, add its counts.
3. When the word changes, print the previous word and its total, then start a new total.
4. After the last line, print the last word and its total.

4. Program

mapper.py

```
#!/usr/bin/env python3
"""Word count mapper.
In : lines of text on stdin
Out: word \t 1   for every word (lower-cased, punctuation removed)
"""
import re
import sys

for line in sys.stdin:
    for word in re.findall(r"[a-z0-9']+", line.lower()):
        print(f"{word}\t1")
```

reducer.py (also used as the combiner)

```
#!/usr/bin/env python3
"""Word count reducer (also used as the combiner).
In : word \t count   sorted by word, so equal words arrive on consecutive lines
Out: word \t total count
"""
import sys

current, total = None, 0
for line in sys.stdin:
    word, count = line.rstrip("\n").split("\t", 1)
    if word != current:
        if current is not None:
            print(f"{current}\t{total}")
        current, total = word, 0
    total += int(count)
if current is not None:
    print(f"{current}\t{total}")
```

5. Execution

Locally (Windows, macOS or Linux, Python 3 only). Both commands produce the same result:

```
$ python run_demo.py
$ cat data/input.txt | python3 mapper.py | sort | python3 reducer.py
```

On Hadoop (run_hadoop.sh):

```
$ hdfs dfs -put data/input.txt /user/$USER/wordcount/input/
$ hadoop jar $HADOOP_HOME/share/hadoop/tools/lib/hadoop-streaming-*.jar \
    -files mapper.py,reducer.py \
    -mapper "python3 mapper.py" -combiner "python3 reducer.py" -reducer "python3
reducer.py" \
    -input /user/$USER/wordcount/input -output /user/$USER/wordcount/output
$ hdfs dfs -cat /user/$USER/wordcount/output/part-*
```

6. Output

The output below is the real output of python run_demo.py. It shows the data after every phase.

Step 1: Input file (10 lines)

```
$ cat data/input.txt
Hadoop is an open source framework for storing and processing big data.
Hadoop stores data in HDFS and processes data with MapReduce.
MapReduce splits a job into map tasks and reduce tasks.
The map task reads input data and emits key value pairs.
The framework sorts the pairs by key and sends every key to one reducer.
The reduce task adds the values of each key and writes the final output.
Word count is the classic first MapReduce program.
In word count the map task emits each word with the value 1,
and the reduce task adds the ones to count how often each word appears.
Big data needs many machines, and MapReduce moves the program to the data.
```

Step 2: Mapper output (word TAB 1) (119 lines)

```
$ python mapper.py < data/input.txt
hadoop    1
is    1
an    1
open    1
source    1
framework    1
for    1
storing    1
... (111 more lines)
```

Step 3: Shuffle and Sort (equal words become adjacent) (119 lines)

```
$ ... | sort
1    1
a    1
adds    1
adds    1
an    1
and    1
and    1
and    1
... (111 more lines)
```

Step 4: Combiner output (local partial counts) (61 lines)

```
$ ... | sort | python reducer.py   (as combiner)
1    1
a    1
adds    2
```

```
an    1
and    8
appears    1
big    2
by    1
... (53 more lines)
```

Step 5: Reducer output (final count for every word) (61 lines)

```
$ ... | sort | python reducer.py
1    1
a    1
adds    2
an    1
and    8
appears    1
... (55 more lines)
```

Complete reducer output (61 distinct words, alphabetical, read down each column):

Word	Count	Word	Count	Word	Count
1	1	in	2	program	2
a	1	input	1	reads	1
adds	2	into	1	reduce	3
an	1	is	2	reducer	1
and	8	job	1	sends	1
appears	1	key	4	sorts	1
big	2	machines	1	source	1
by	1	many	1	splits	1
classic	1	map	3	stores	1
count	3	mapreduce	4	storing	1
data	6	moves	1	task	4
each	3	needs	1	tasks	2
emits	2	of	1	the	13
every	1	often	1	to	3
final	1	one	1	value	2
first	1	ones	1	values	1
for	1	open	1	with	2
framework	2	output	1	word	4
hadoop	2	pairs	2	writes	1
hdfs	1	processes	1		
how	1	processing	1		

Step 6: Top 10 most frequent words (10 lines)

```
$ sort -k2,2nr output/wordcount.txt | head -10
the    13
and    8
data    6
key    4
mapreduce    4
task    4
word    4
count    3
each    3
map    3
```

Step 7: Summary and verification (4 lines)

```
$ python run_demo.py
Total words (mapper records) : 119
Distinct words (reducer keys): 61
Records after combiner       : 61
Matches a direct Counter()  : YES
```

7. Result

The word count program was written in Python using the MapReduce model and run successfully. The mapper turned the 10-line input into 119 (word, 1) pairs. After the shuffle and sort, the reducer added the 1s for each of the 61 distinct words. The most frequent word is “the”, followed by “and” and “data”. The result matches a direct count made without MapReduce.